2022 IEEE 20th International Conference on Embedded and Ubiquitous Computing (EUC) | 979-8-3503-9635-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/EUC57774.2022.00010

# CARET: Chain-Aware ROS 2 Evaluation Tool

Takahisa Kuboichi
*Graduate School of Science and Engineering*
*Saitama University*

Atsushi Hasegawa
*Research Institute of Systems Planning*

Bo Peng and Keita Miura
*EMB IV*

Kenji Funaoka
*TIER IV*

Shinpei Kato
*Graduate School of Information Science and Technology*
*The University of Tokyo*

Takuya Azumi
*Graduate School of Science and Engineering*
*Saitama University*

***Abstract*—This paper presents a tool to evaluate the latency of Robot Operating System (ROS) 2 applications called Chain-Aware ROS 2 Evaluation Tool (CARET). ROS 2 is designed to enhance the modularity of real-time robotic applications, including self-driving software such as Autoware. To analyze the performance of ROS 2 applications, CARET supports measurement functionalities for the callback latency, node latency, communication time between nodes, and end-to-end latency. To calculate each latency, CARET provides tracepoints, an architecture file, information on tracepoint connections, and a message tracking functionality. Furthermore, CARET can visualize different types of latency, such as bottlenecks and lost message, to analyze ROS 2 applications. The experimental results demonstrate that CARET can successfully measure the end-to-end latency of Autoware.Universe, which is ROS 2-based self-driving software.**

***Index Terms*—ROS 2, Self-driving Systems, End-to-end Latency**

## I. Introduction

Self-driving technology is attracting attention worldwide. Autonomous vehicles have a lower accident rate than humans [1] and thus have the potential to reduce the number of accidents. In addition, they can provide a means of transportation, and logistics to underpopulated areas. In the future, autonomous vehicles may be able to further reduce the accident rate by receiving sensor information not only from their own sensors but also from the sensors of other vehicles. Autonomous vehicles have many additional benefits, such as the ability to reduce traffic congestion.

Autonomous vehicles [2] are used to be developed using the Robot Operating System (ROS) [3], which is the de facto standard for robotic applications. ROS is composed of processing units called nodes, each of which executes multiple callbacks. ROS is characterized by a wide range of libraries, tools, distributed processing, and the ease of adding new functionalities. However, despite its advantages, ROS has drawbacks, one of which is that it does not meet real-time requirements [4] [5]. Lacking real-time capability is one of the serious problems for autonomous vehicles. If a vehicle does not promptly stop when it senses danger, an accident may occur. To avoid accidents, ROS must satisfy real-time requirements.

ROS 2 [6] [7] has been developed to solve the problems of ROS, and it inherits the advantages of ROS, while resolving the problem of real-time performance. ROS 2 applications follow a processing chain. A chain represents a set of nodes in a ROS 2 application that can take through publish/subscribe communication. ROS 2 has begun to be employed in self-driving software [8]. Self-driving software must identify areas where there are processing delays. These areas are the bottlenecks of the system and must be improved; otherwise, high latency may cause accidents. Vehicle accidents can cause loss of life and must be avoided at all costs. To increase safety, the system must perform in real-time; that is, processing must be completed within a certain period of time.

Self-driving software can guarantee safe driving if all processing is completed within a deadline [8]. Self-driving software receives data from sensors, starts processing the data, and finally issues commands to move the vehicle. To demonstrate that the system operates in real-time, the processing time of the application must be measured and evaluated. However, when inserting the time measurement code into the application for evaluation, ROS 2 application developers must modify the time measurement code if the application changes, which delays the development. To evaluate an application without slowing down the development process, the latency of ROS 2 applications must be measured without modifying the code.

Since ROS 2 has been recently developed, there are insufficient tools to evaluate the performance of ROS 2 applications. For example, existing tools cannot measure the accurate end-to-end latency of ROS 2 applications[1]. In this paper, end-to-end latency of self-driving software is defined as the time from the receiving sensing data to the control. Accurate end-to-end latency is important to ensure real-time performance. Without a guarantee that the end-to-end processing can be performed by a certain deadline, safe self-driving software cannot be developed. ROS 2 applications that must operate in real-time must accurately measure the end-to-end latency, and a convenient tool that can measure latency is required.

This paper proposes the Chain-Aware ROS 2 Evaluation Tool (CARET), which is a tool that evaluates the latency of ROS 2 applications. CARET is capable of measuring the callback latency, node latency, communication time between nodes, and end-to-end latency. In addition, CARET tracks the message flow by examining ROS 2 chains and measures the application latency. By combining information on each latency

[1]Some existing tools can measure end-to-end latency of ROS 2 in case of the simple structure of an application such as a single rate.

979-8-3503-9635-5/22/$31.00 ©2022 IEEE
DOI 10.1109/EUC57774.2022.00010



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.

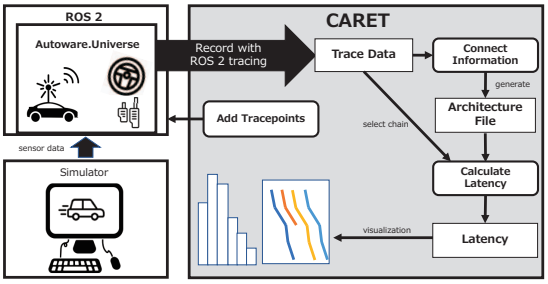


Fig. 1. System model.

and the message flow, CARET can measure the end-to-end latency.

In summary, this paper offers the following contributions:

- End-to-end latency measurement: CARET calculates the end-to-end latency of any chain and creates a time series and histogram of the end-to-end latency. By using CARET, developers can determine whether self-driving software meets the deadlines of ROS 2 applications.
- Message flow tracking: By tracking communication messages between nodes, the calculation accuracy can be improved. By tracking the message flow, we can find lost messages which are messages that never arrived, and the end-to-end latency can be calculated more accurately than by existing measurement tools.
- ROS 2 application analysis: CARET visualizes the latency of each node and the communication latency between nodes. By analyzing ROS 2 applications with CARET, users can identify lost messages.

The remainder of this paper is organized as follows. Section II introduces ROS 2 and Autoware.Universe, a set of practical ROS 2 applications for self-driving software, and introduces existing tools that were enhanced to perform measurements. Section III describes the proposed evaluation tool. Section IV evaluates Autoware.Universe and the overhead of CARET and ROS 2 tracing. Section V describes related work and compares them with the present work. Finally, Section VI concludes the paper.

## II. System Model

The system model of CARET is presented in Fig. 1. CARET is a measurement tool for ROS 2 applications and can measure Autoware.Universe. CARET uses the next-generation Linux Trace Toolkit: (LTTng) [9] and ROS 2 tracing [10] for measurements.

ROS 2 tracing measures the latency of ROS 2 applications by using tracepoints to determine when processes are executed. CARET generates an architecture file by extracting node data from the trace data. The architecture file contains static information such as the node name, callback name, and publish/subscribe information. By connecting the architecture file and trace data, the chain latency can be calculated. The chain is defined by the user. CARET visualizes the latency as a graph that contains information such as where messages were lost and how long the chain latency was.

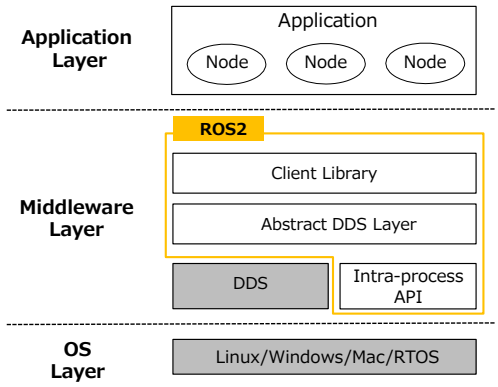


Fig. 2. ROS 2 structure.

The system model of the proposed framework is also introduced in this section. First, ROS 2, an open-source robotics middleware is outlined. Next, we describe Autoware.Universe [11] [12] [13], open-source self-driving software.

### A. ROS 2

ROS 2, a platform for developing robotic applications, is used worldwide, such as to develop self-driving software. The structure of ROS 2 is presented in Fig. 2 and consists of three layers: an application layer, a middleware layer, and an operating system (OS) layer.

In the application layer, nodes exist independently, and each node is executed as a separate process, thereby enabling distributed processing. This structure allows ROS 2 to add additional nodes without affecting other existing nodes. A node in ROS 2 executes multiple callbacks, which are the smallest execution units in ROS 2. In the middleware layer, the Data Distribution Service (DDS) is implemented, which is a communication method developed separately from ROS 2. DDS allows users to emphasize either real-time performance or communication reliability by setting a parameter called Quality of Service. DDS is a reliable technology that has been used in systems such as financial and aerospace systems. In the OS layer, various operating systems, such as Linux, Windows, macOS, and Real-Time OS (RTOS), are supported. With support for many operating systems, ROS 2 is expected to be widely utilized.

ROS 2 uses the publish/subscribe model shown in Fig. 3 to communicate between nodes. A node is an executable process, and a topic stores the data to be exchanged by nodes. This method enables one-to-one and one-to-many communication. A node publishes data to a topic. In Fig. 3, the camera node publishes image data to a topic, while the traffic light recognition, and pedestrian recognition node subscribe to the topic.



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.

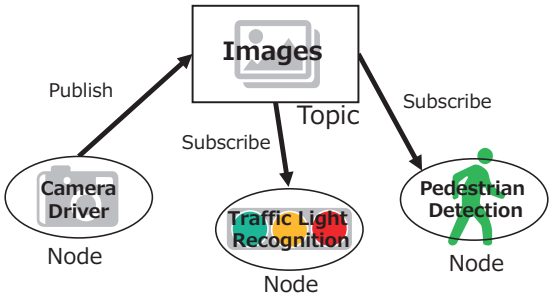


Fig. 3. Publish/subscribe model.

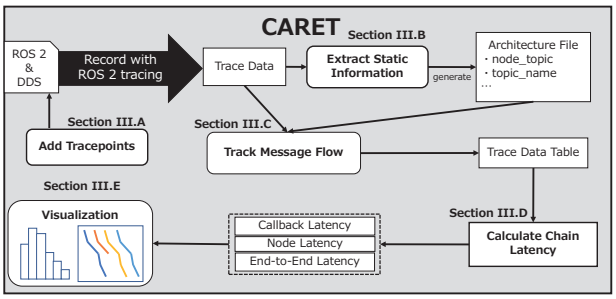


Fig. 4. CARET structure.

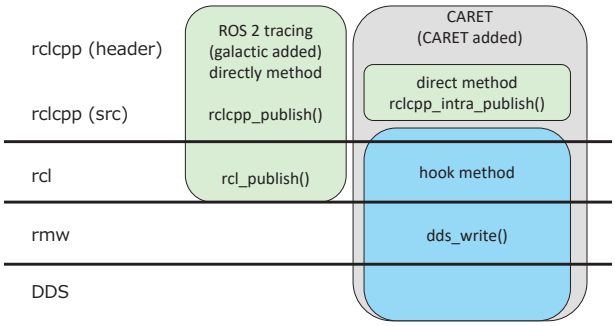


Fig. 5. Tracepoints added ROS 2 layers.

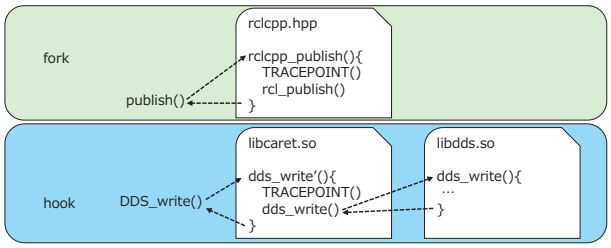


Fig. 6. Hooking with LD_PRELOAD.

### B. Autoware.Universe

Autoware.Universe is open-source self-driving software based on ROS 2. Autoware.Universe provides a rich set of self-driving modules, such as localization, planning, and control. The localization module is based on scan matching with 3D maps and LiDAR scanners to estimate the position and rotation. The path planning module generates the path to the destination. The control module calculates the velocity and angle to follow the path.

### C. ROS 2 Tracing

ROS 2 tracing [10] is a tool developed for ROS 2 based on LTTng [9]. ROS 2 tracing can record runtime data, system data, and user-defined data. ROS 2 tracing allows ROS 2 frameworks to retrieve user-defined data, such as the time at which a message arrives, a timer fires, or a callback is executed. When a publisher, subscriber, service, or client is created, ROS 2 tracing must remember the correct reference to distinguish the tracepoints from other tracepoints for the same instance.

## III. CARET: Chain-Aware ROS 2 Evaluation Tool

The structure of CARET is presented in Fig. 4. CARET extracts information from the trace data, which include the tracepoint information originally included in ROS 2 and the tracepoints added by CARET. By connecting the tracepoint information, the architecture file is created, which contains information such as callbacks, topic names, and node names. By selecting the chain that the user wishes to measure in the architecture file, the user can calculate the latency of the target chain. The calculated latency can then be visualized for analysis.

### A. Adding Tracepoints

In Fig. 5, each layer of the ROS 2 library is displayed. CARET adds more tracepoints than originally implemented in the ROS 2 Galactic distribution. The tracepoints originally implemented in ROS 2 Galactic included rclcpp_publish() and callback_start(). In CARET, there are two methods to add tracepoints that are missing in the measurement. The first method is to rewrite the ROS 2 library directly, as in ROS 2 Galactic. This method adds tracepoints to functions such as rclcpp_intra_publish() and has the advantage of being able to add tracepoints to any function.

Since CARET uses more tracepoints than ROS 2 tracing, such as dispatch_subscription_callback, the range that can be measured, such as the communication time between nodes, is increased. The second method to add missing tracepoints is the hook method using LD_PRELOAD. This method adds a tracepoint to a function such as dds_write(). Since dds_write() is added by the hook, it does not directly rewrite the ROS 2 library. Although this method cannot be added to functions defined in the header, maintenance is simple because it is not necessary to make any code changes even if minor changes to ROS 2 are made.

The method of compiling a function by adding a tracepoint is illustrated in Fig. 6. In the function rclcpp_publish(), which was originally implemented in the tracepoint pro-



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.

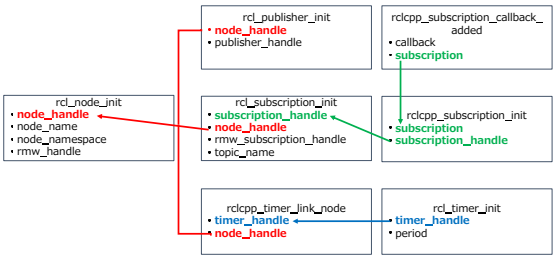


Fig. 7. Connecting tracepoint information.

```
...
node_chain:                                          Chain Information
 - node_name: /sensor_dummy_node
   publish_topic_name: /topic1
   subscribe_topic_name: UNDEFINED
 - node_name: /filter_node
   publish_topic_name: /topic2
   subscribe_topic_name: /topic1
   ...
- node_name: /sensor_dummy_node                      Node Information
  callbacks:
  - callback_name: /sensor_dummy_node/callback_0
    callback_type: timer_callback
    period_ns: 100000000
    ...
  publishes:
  - topic_name: /topic1
    callback_names:
    - /sensor_dummy_node/callback_0
    ...
- node_name: /filter_node                            Node Information
  callbacks:
  - callback_name: /filter_node/callback_0
    callback_type: subscription_callback
    topic_name: /topic1
    ...
  publishes:
  - topic_name: /topic2
    callback_names:
    - /filter_node/callback_0
  subscribes:
  - topic_name: /topic1
    callback_name: /filter_node
```

Fig. 8. Architecture file.

cess is included in the function rclcpp_publish(). In the function dds_write() where the tracepoint is added by the hook, the function dds_write() defined by LD_PRELOAD is searched with priority. The function dds_write() defined by LD_PRELOAD contains the tracepoint and original function dds_write(). The original function dds_write() is searched when the compiler reads the LD_PRELOAD function. As a result, the original function with tracepoints is compiled. Since Autoware_Perf [12] only uses the method of rewriting the ROS 2 library, the tracepoints must be defined even when minor changes are made to the ROS 2 library.

### B. Connecting Tracepoint Information

How tracepoints information is connected is illustrated in Fig. 7. The following procedure is used to link callbacks to node names: (1) CARET obtains callback information with rclcpp_subscription_callback_added(), (2) CARET links rclcpp_subscription_callback_added() with subscription information in rclcpp_subscription_init(), (3) CARET links rclcpp_subscription_init() and rcl_subscription_init() with information from subscription_handle(), and (4) CARET links rclcpp_subscription_init() and rcl_node_init() by information from node_handle().

By connecting the tracepoint information, node information, such as the callback, publisher handle, subscription handle, and topic name, can be obtained. CARET generates the architecture file using the node information. The architecture file[2] is illustrated in Fig. 8 and has the following features. It contains node information and defines the chain to be measured. In addition, it specifies the method of calculating the node latency. The architecture file is automatically generated from the trace data of CARET. The information that can be obtained at initialization and runtime is used to connect tracepoint information.

Autoware_Perf [12] requires users to generate the architecture file. In addition, the architecture file does not have information about the connections between messages. For this reason, Autoware_Perf calculates the estimated end-to-end latency by convolutional integration instead of actual measurement. By using CARET, the number of tracepoints is increased for a more detailed analysis than Autoware_Perf, and messages can be connected by making changes to the architecture file. The actual measured end-to-end latency can be measured, and lost messages which are messages that never arrived can be identified by connecting messages.

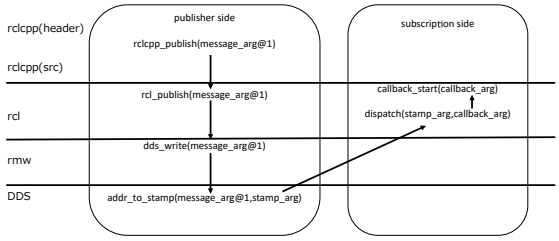


Fig. 9. How to track process-to-process message flow.

### C. Tracking Message Flow

CARET tracks the message flow to measure the exact latency. The method of tracking the message flow is presented in Fig. 9. CARET connects the tracepoint information from the trace data and the architecture file. The trace data contain tracepoint information, while the architecture file contains chain information. CARET tracks the flow of a message as illustrated in Fig. 9. Tracepoints rclcpp_publish(), rcl_publish(), dds_write(), and addr_to_stamp() contain message_arg@1 variable. CARET uses message_arg@1 variable to connect these tracepoints. CARET also uses the stamp_arg variable to connect tracepoints addr_to_stamp() and dispatch(). In addition, CARET uses the callback_arg variables to connect tracepoints dispatch() and callback_start(). As a result, CARET tracks message flow. Since Autoware_Perf does not have the ability to track the message flow, the exact time at which the application node publishes subscription messages

[2]The full architecture file of the CARET demo can be found at this URL https://github.com/tier4/CARET_demos/blob/main/samples/end_to_end_sample/architecture.yaml.



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.

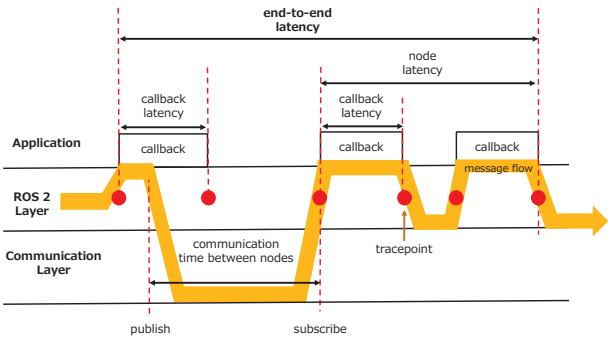


Fig. 10. Message flow and tracepoints.

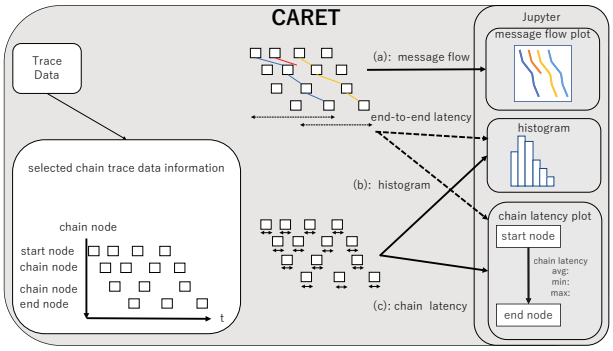


Fig. 11. Latency visualization.

cannot be determined. In contrast, CARET users can determine the exact time at which a node publishes a subscription message.

### D. Latency Calculation

The location of the tracepoints when calculating latency is illustrated in Fig. 10. Tracepoints are inserted at the start and end of the callback to measure the callback latency. To measure the communication latency, a tracepoint is inserted when a callback is published, and another tracepoint is inserted when the next callback subscribes. Using these tracepoints, the callback and communication latency can be calculated.

CARET performs two types of latency calculations. The first type calculates the latency of inter-process communication. The tracepoints are linked as described in the chapter on linking tracepoints. As a result, since rclcpp_publish() and callback_start() can be linked, calculate the time using the time of callback_start() minus the time of rclcpp_publish(). The second is node latency. CARET calculates node latency by using the callback chain, which is shown in Fig. 16. In the method of using the callback chain, the latency is calculated by the difference between callback_start() and rclcpp_publish() with the same callback address. Callback_start() and rclcpp_publish() are linked to the nearest callback_start from the rclcpp_publish(). This method has the advantage that it does not depend on the message type. In addition, the implementation of the ROS 2 application does not need to be changed if the latest message is used between callbacks.

### E. Latency Visualization

CARET visualizes the latency to analyze ROS 2 applications. The latency visualization is illustrated in Fig. 11. CARET has three main methods of visualizing a selected chain. All visualization methods use Jupyter [14] to display results in a browser quickly and conveniently.

The first method is to output the chain latency as a histogram. In this histogram, a CARET user can see the frequency of the latency. In this method, CARET calculates the chain's latency in the Jupyter browser and creates a histogram data frame. The created data frame is passed to the Bokeh library, which visualizes the histogram. By calculating the chain latency, ROS 2 application developers can evaluate the applications that require strict deadlines, such as self-driving software.

The second visualization method is to visualize the chain latency as a table. In this method, the minimum, average, and maximum values of the latency each process in the table are obtained. This method calculates the latency of each process and outputs the results as a table. This visualization method is not available in Autoware_Perf.

The third method is visualization using the message flow. In the message flow, messages are linked using the trace data, and a single line indicates how much time is spent in each process. Each line depicts the flow of the application chain and is colored to help distinguish the differences of each message flow. By connecting the chain, the actual message flow is visualized. This method visualizes the x-axis at time, and the y-axis as each process. The visualized graph can zoom in and out. To analyze a few processes, the CARET user recommended zooming in the y-axis. To analyze all chains in the specified time, the CARET user recommended zooming in the x-axis. Since Autoware_Perf lacks this type of diagram, it cannot detect lost data and examine whether the lost data changed the behavior of the application. In contrast, CARET can examine the data and perform detailed analysis.

TABLE I
EVALUATION PLATFORM

| | |
|---|---|
| Processor | Intel Core-i7-11800 CPU @ 2.3 GHz |
| Number of Cores | 8 |
| Memory | 16 GB |
| GPU | NVIDIA GeForce RTX 3050 |
| OS | Ubuntu 20.04 |
| ROS 2 | Galactic |

## IV. EVALUATION

### A. Autoware.Universe Evaluation

Autoware.Universe was measured using CARET. The environment used for the measurements is summarized in Table I. The same platform was used for the measurements



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.

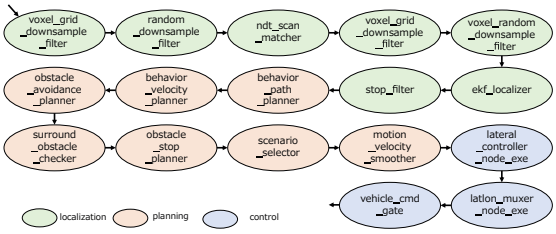


Fig. 12. End-to-end Autoware.Universe chain.

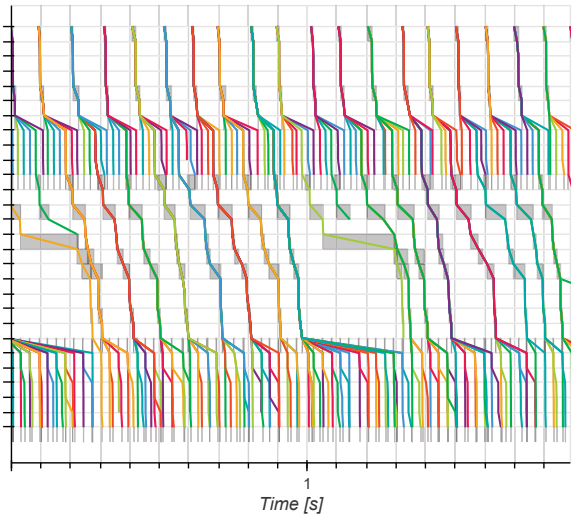


Fig. 13. Autoware.Universe end-to-end message flow.

in the following evaluations. The end-to-end latency of Autoware.Universe from the localization node to the control node was measured. The measured end-to-end chain is shown in Fig. 12. Sensing nodes read and process the information around the vehicle from LiDAR, while localization nodes estimate the position of the vehicle by combining the information around the vehicle and the information on the map. Planning nodes determine how the vehicle will move based on the vehicle position and information about surrounding cars, pedestrians, and traffic lights. Control nodes issue commands to move the vehicle.

The Autoware.Universe end-to-end message flow is presented in Fig. 13. The x-axis represents the simulation time, while the y-axis represents the events that occur when a node is executed. The colored lines in Fig. 13 represent the message flow of the application, where different messages are indicated by different colors. Lines disappear when messages are lost. In addition, the gray rectangles indicate the node latency. When hovering over a diagram, CARET displays a table where the user can see detailed information about each process. The table provides information such as the start and end times of the process as well as the latency. According to Fig. 13, Autoware.Universe has high latency for a particular process; namely, the crop_box_filter node takes

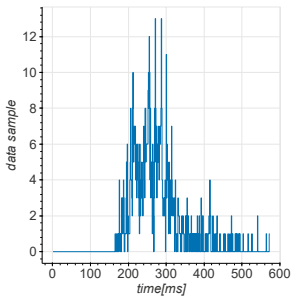


Fig. 14. End-to-end histogram of Autoware.Universe.

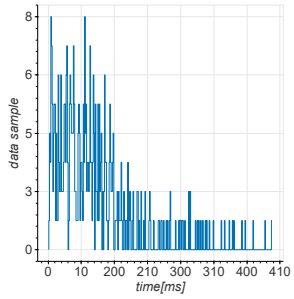


Fig. 15. Lateral controller node latency.

more time to process than other nodes. This node processes LiDAR information around the vehicle. The cause of high latency should be investigated to improve the performance of Autoware.Universe. The ekf_localizer node has multiple lines in one message because this node publishes multiple times to subsequent nodes. The obstacle_avoidance_planner node, which avoids obstacles, occasionally has high latency. Autoware.Universe developer needs to investigate the reason for the increased latency needs to be investigated further.

The end-to-end latency as a histogram is presented in Fig. 14. The x-axis of the histogram represents the time in milliseconds, while the y-axis represents the latency of the data sample. In this case, to avoid the effect of the cache, the time from 30 seconds after the start of the measurement is visualized. Most of the data are between 200 and 350 ms; however, some exceed 500 ms. The end-to-end latency required for safe self-driving software is 100 ms. However, in this measurement, most of the results exceed 100 ms. This result is due to the performance of the processor evaluation used was not enough. Research on many-core [2] [7] and SoC [15] is progressing, and we hope that this will lead to the development of higher performance hardware and improved execution time. To improve the end-to-end latency, improving bottlenecks is important. CARET can be used to identify bottlenecks in Autoware.Universe, as it can calculate any chain in a ROS 2 application. By improving the bottleneck performance, the end-to-end latency can be improved.

The latency of lateral_controller node latency is presented in Fig. 15. This node has one of the the high latency nodes of the Autoware.Universe end-to-end latency; therefore, it may be a bottleneck in Autoware.Universe. This node takes 72.91 ms on average. The maximum latency of the lateral_controller node is 338.95 ms. Therefore, it may be necessary to reduce the time it takes for this node to execute.

The chain latency of end-to-end chain Latency is presented in Fig. 16. The chain latency table presents the minimum, average, and maximum latency of the nodes as well as the communication time between nodes. This functionality is convenient for identifying bottleneck nodes in the application.



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.

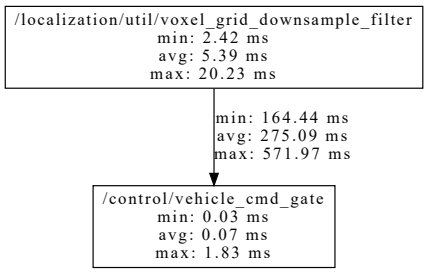


Fig. 16. End-to-end chain latency.

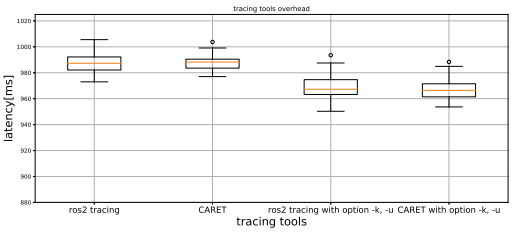


Fig. 17. Publish and subscribe without timer execution time with each tracing tool.

### B. Measuring Overhead

To measure the overhead of CARET, the communication time latency of ROS 2 applications was measured without tracing, with ROS 2 tracing, and with CARET. In this evaluation, the execution time was measured with two options. The first option was -k, which traced only the kernel space, while the second option was -u, which traced only user applications. To measure the overhead, talker and listener, simple applications in ROS 2 were used. Talker and listener originally had a timer that performed publishing and subscribing every second; however, the timer was not used in the evaluation to ensure that the overhead of the tracing tool was appropriate. If the nodes were executed every second, the tracing tool might have been executed while waiting for the timer. In addition, to ensure the tracing tools, talker, and listener are running in the same core, the number of cores was limited to one. The size of the message to be published was one byte.

The execution time results for 10,000 publish and subscribe runs are presented in Fig. 17. Without options -k and -u, CARET and ROS 2 tracing take approximately around 970 to 1,010 ms. With options -k and -u, CARET and ROS 2 tracing take approximately 950 to 995 ms. Compared to the execution time of ROS 2 tracing, the execution time of CARET is almost identical both with and without the options. Since the tracing procedure itself is executed in a separate thread, there is practically no overhead in execution time for the ROS 2 application, except when CPU utilization is close to 100%. In other words, CARET does not cause overhead, in contrast to using ROS 2 tracing.

### C. Lessons Learned

CARET was used to measure Autoware.Universe, and the results indicated that the end-to-end latency of Autoware.Universe is over 100 ms in the environment in Table I. There are two possible solutions for developers to ensure the safety of Autoware.Universe. The first is to use higher-performance hardware than the laptop computer used in this evaluation, while the second is to streamline the processing of Autoware.Universe. The results of the Autoware.Universe measurements were unexpected. Since Autoware and other self-driving software are set to not lose messages, it was predicted that almost no messages would be lost. The results indicated that the ekf_localizer node lost many of the messages. Upon investigation, however, it was found that messages were not actually lost. The ekf_localizer node was publishing to many nodes and thus publishing at a higher rate than the subscribers on the chain. This process resulted in what appeared to be lost messages. CARET can allow users to be able to find not only ROS 2 application bottlenecks but also nodes that publish more than necessary.

In the design phase, this study attempted to measure all the nodes by considering the callback chain in ROS 2 applications; however, some nodes could not be measured. Some Autoware.Universe nodes received messages, stored them in buffers, and then published the messages. In this case, the node latency could not be calculated using the callback chain method. There are two methods of calculating the latency of a node in this case. The first is to change the implementation of Autoware.Universe to measure it directly. However, changing the implementation of ROS 2 applications is not desirable as it reduces the maintainability. The second method is to record only the node latency without measuring the latency of the callbacks in the node. One way to calculate only the node latency is to use the header timestamps sent and received by the nodes. In this method, the node measurement is less accurate than in the method of using callback chains; however, this method can measure cases in which buffers are used. A demonstration movie of CARET for Autoware.Universe nodes, can be found at the following **hyperlink: https://youtu.be/hHhi9mP4LNg**.

## V. Related Work

ROSCH [16] is a real-time scheduling framework for ROS whose real-time performance has been evaluated to ensure that the end-to-end latency is less than a certain value. Autoware on Board [11] is an end-to-end latency measurement tool to evaluate Autoware.AI, which was the previous version of Autoware. In addition, RAPLET [17] was developed to evaluate ROS applications. RAPLET measures the callbacks latency, node latency, communication latency, and kernel time. However, these existing tools only support ROS.

LTTng [9] is an open-source software toolkit that simultaneously traces the Linux kernel, user applications, and user libraries. However, LTTng was developed for Linux. Apex.AI_performance_test tool [18] measures latency, CPU



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF CARET AND EXISTING MEASUREMENT TOOLS

| | ROS 2 appplications | Latency analysis | End-to-End latency | Track the flow of ROS 2 applications |
|---|---|---|---|---|
| LTTng [9] | ✓ | ✓ | | |
| ROS 2 tracing [10] | ✓ | ✓ | | |
| Apex.AI performance_test [18] | ✓ | ✓ | | |
| buildfarm_perf_tests [19] | ✓ | ✓ | | |
| Autoware on board [11] | | ✓ | ✓ | |
| iRobot ROS 2_performance [20] | ✓ | ✓ | | |
| Autoware_Perf [12] | ✓ | ✓ | ✓ | |
| CARET | ✓ | ✓ | ✓ | ✓ |

usage, maximum RAM virtual page size, page faults and context switches of ROS 2. However, Apex.AI_performance_test cannot measure a topic that has multiple publishers. Open Robotics buildfarm_perf_tests tool [19] measures the average latency, CPU utilization, the number of send and receive packets per second, the number of lost packets, and maximum resident set size of rmw. However, this tool cannot measure the latency between different rmw implementations. iRobot can measure specified nodes and topics; however, it cannot measure customized nodes.

ROS 2 tracing measures and analyzes the trace data, including the callback duration, message elapsed time, message size, and memory usage. However, ROS 2 tracing [10] cannot measure the node latency, communication time between nodes, DDS, or end-to-end latency. Autoware_Perf [12] can measure the callback latency, node latency, and communication latency in ROS 2 applications. In addition, Autoware_Perf calculates the end-to-end latency by a convolutional integral as an estimated value. To the best of out knowledge, the existing performance measurement tools cannot meet all these requirements as showed in Table II. For this reason, CARET (Chain-Aware ROS Evaluation Tool), which can be adapted to any ROS 2 applications, was proposed.

## VI. CONCLUSION

This paper proposed Chain-Aware ROS 2 Evaluation Tool (CARET) to evaluate the latency ROS 2 applications. CARET can measure the ROS 2 callback latency, node latency, communication time between nodes, and end-to-end latency. This capability allows developers to identify bottlenecks in ROS 2 applications. We evaluated Autoware.Universe using CARET and identified the bottlenecks in Autoware.Universe and their causes. The measurement results indicated that the end-to-end latency of Autoware.Universe is approximately 300 ms. Safe driving is ensured if all processes are executed within 100 ms. There are two options for developers to ensure the safety of Autoware.Universe. The first is to use higher-performance hardware than what was used in the evaluation, while the second is to improve the bottleneck performance of Autoware.Universe.

CARET has challenges; namely, it cannot measure nodes that use buffers. To measure such nodes, CARET will be improved in future work to measure the node latency. Specifically, as almost all messages have a header timestamp, the header timestamp can be used to calculate the node latency. CARET is all open-source and can be downloaded from GitHub **https://github.com/tier4/CARET**.

## ACKNOWLEDGMENT

This work was partially supported by the New Energy and Industrial Technology Development Organization (NEDO).

## REFERENCES

[1] J. B. Greenblatt and S. Shaheen, "Automated vehicles, on-demand mobility, and environmental impacts," *Current sustainable/renewable energy reports*, vol. 2, no. 3, pp. 74–81, 2015.
[2] T. Azumi, Y. Maruyama, and S. Kato, "ROS-lite: ROS Framework for NoC-based Embedded Many-core Platform," in *Proc. of IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020*. IEEE, 2020, pp. 4375–4382.
[3] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng, "ROS: an open-source Robot Operating System," in *Proc. of IEEE International Conference on Robotics and Automation, Open-Source Software Workshop*, 2009.
[4] J. Park, R. Delgado, and B. W. Choi, "Real-time characteristics of ROS 2.0 in multiagent robot systems: An empirical study," *IEEE Access, 2020*, vol. 8, pp. 154 637–154 651, 2020.
[5] Z. Jiang, Y. Gong, J. Zhai, Y.-P. Wang, W. Liu, H. Wu, and J. Jin, "Message Passing Optimization in Robot Operating System," *International Journal of Parallel Programming, 2020*, vol. 48, no. 1, pp. 119–136, 2020.
[6] Y. Maruyama, S. Kato, and T. Azumi, "Exploring the Performance of ROS2," 2016, pp. 1–10.
[7] T. Shuhei and T. Azumi, "ROS 2 Framework for Embedded Multi-core Platform," in *Proc. of Asia Pacific Conference on Robot IoT System Development and Platform (APRIS)*, 2021.
[8] M. Reke, D. Peter, J. Schulte-Tigges, S. Schiffer, A. Ferrein, T. Walter, and D. Matheis, "A self-driving car architecture in ROS2," in *Proc. of Robotics and Mechatronics Conference*. IEEE, 2020, pp. 1–6.
[9] "LTTng," 2022. [Online]. Available: https://lttng.org/
[10] "ROS 2_tracing," https://index.ros.org/r/ros2_tracing/.
[11] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *Proc. of ACM/IEEE International Conference on Cyber-Physical Systems (ICCPS)*, 2018, pp. 287–296.
[12] Z. Li, A. Hasegawa, and T. Azumi, "Autoware_Perf: Tracing and Performance Analysis Framework for ROS 2 Applications," *Elsevier Journal of Systems Architecture*, 2022.
[13] T. Miki and T. Azumi, "Porting Evaluation Framework for Self-Driving Software Based on ROS 2," in *Proc. of Asia Pacific Conference on Robot IoT System Development and Platform (APRIS)*, 2021.
[14] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, and C. Willing, "Jupyter notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds. IOS Press, 2016, pp. 87 – 90.
[15] H. Chishiro, K. Suito, T. Ito, S. Maeda, T. Azumi, K. Funaoka, and K. Shinpei, "System on a Chip and Software Platforms for Fully Autonomous Driving," in *Proc. of IEEE International Conference on Embedded Software and Systems, Nevada, USA*, 2019.
[16] Y. Saito, F. Sato, T. Azumi, S. Kato, and N. Nishio, "ROSCH: Real-time scheduling framework for ROS," in *Proc. of IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2018.
[17] K. Nishimura, T. Ishikawa, H. Sasaki, and S. Kato, "RAPLET: Demystifying Publish/Subscribe Latency for ROS Applications," in *Proc. of IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), 2021*. IEEE, 2021, pp. 41–50.
[18] "ApexAI: performance test," 2022. [Online]. Available: https://gitlab.com/ApexAI/performance_test/
[19] "Open Robotics: buildfarm perf tests," 2022. [Online]. Available: https://github.com/ros2/buildfarm_perf_tests
[20] "iRobot: ROS2 performance," 2022. [Online]. Available: https://github.com/irobot-ros/ros2-performance



Authorized licensed use limited to: KU Leuven University Library. Downloaded on January 27,2026 at 22:18:12 UTC from IEEE Xplore. Restrictions apply.